MAESTRÍA EN ARQUITECTURA DE SOFTWARE · UNIVERSIDAD DE LA SABANA

Documento de Arquitectura de Software

Sistema de Soporte Decisional para la Reducción del Desperdicio Alimentario en Bancos de Alimentos

Caso: Banco Diocesano de Alimentos de Neiva

PARA:

Jenny Alexandra Múnera Serna

Asesora · Maestría en Arquitectura de Software, Universidad de La Sabana

POR:

Luis Alberto Rojas Adames
Astrid Carolina Rodríguez Cristancho
Alexander Caballero García

Chía, Cundinamarca · 2026

Control del Documento

Versión	Fecha	Cambiado por	Comentarios
1.0	2026-10-11	Luis Alberto Rojas Adames, Astrid Carolina Rodríguez Cristancho, Alexander Caballero García	Versión inicial del documento de arquitectura.

Tabla de Contenido

Introducción 5

1. Contexto de la Solución 5

1.1 Situación Actual 5

1.2 Objetivo 5

1.3 Vista de Contexto C4 5

2. Drivers Arquitectónicos 6

2.1 Funcionalidades Principales 6

2.2 Restricciones 6

2.3 Supuestos 6

2.4 Atributos de Calidad 7

3. Decisiones de Arquitectura 10

3.1 Estilo Arquitectónico: Monolito Modular 10

3.2 Patrones 11

3.2.1 Clean Architecture 11

3.2.2 Domain-Driven Design 11

3.2.3 CQRS 11

3.2.4 Unit of Work 11

3.2.5 Comunicación entre Módulos mediante Contrato 12

3.2.6 Análisis Disparado por Movimiento con Recálculo Periódico 12

3.2.7 Notificaciones en Tiempo Real con Server-Sent Events 12

3.2.8 Control de Acceso Basado en Roles 12

3.2.9 Registro de Autorizaciones de Tratamiento de Datos 12

3.2.10 Minimización de Datos Personales 12

3.3 Base de Datos 12

3.4 Servicios Cloud Seleccionados 12

3.5 Stack Tecnológico 13

4. Definición de Arquitectura 13

4.1 Vista Funcional 13

4.1.1 Diagrama de contenedores 13

4.1.2 Diagrama de componentes 14

4.2 Vista de Información (Base de Datos) 15

4.3 Vista de Despliegue 16

4.4 Vista de Desarrollo (Paquetes) 17

5. ADRs 17

ADR-001: Estilo arquitectónico 18

ADR-002: Aislamiento de datos por banco (multitenant) 18

ADR-003: Comunicación con el módulo de análisis 19

ADR-004: Registro del inventario....21
ADR-005: Reglas de negocio configurables....21
ADR-006: Importación y exportación de archivos....22
ADR-007: Plataforma de despliegue en AWS....23
ADR-008: Actualización de pantallas en tiempo real....24
ADR-009: Aplicación de FEFO al armar kits....25

Introducción

Este documento describe la arquitectura del Sistema de Soporte Decisional para la reducción del desperdicio alimentario en bancos de alimentos, tomando como caso el Banco Diocesano de Alimentos de Neiva. Se dirige a la asesora del trabajo de grado de la Maestría en Arquitectura de Software de la Universidad de La Sabana y al equipo de desarrollo. Cubre el contexto de la solución, los drivers arquitectónicos, las decisiones de diseño con sus patrones y tecnologías, las vistas funcional, de información, de despliegue y de desarrollo, y las decisiones de arquitectura (ADR) que implicaron un compromiso entre alternativas. El sistema es un prototipo limitado a los flujos críticos de trazabilidad y riesgo de vencimiento, que se valida con escenarios de calidad.

1. Contexto de la Solución

1.1 Situación Actual

El Banco Diocesano de Alimentos de Neiva registra la recepción de donaciones y las entregas a organizaciones en papel y en hojas de Excel, sin lotes ni existencias. Entre enero y junio de 2026 entregó cerca de 357 toneladas en unas 2.400 entregas. Como el formato de entrada no registra lotes y el de salida no conserva la fecha de vencimiento, no es posible relacionar una entrega con la donación de la que salió, no se sabe cuánto se pierde en bodega ni qué está próximo a vencer, y el mismo dato se digita en varios formatos hasta armar a mano el reporte mensual para la red de bancos, cuya estructura además cambia.

Esto afecta a la coordinación, que decide sin información de inventario, y al personal administrativo y de bodega, que redigita la información. El alcance es un prototipo para el banco de Neiva; si el modelo tiene éxito podría extenderse a otros bancos de alimentos, por lo que el diseño queda preparado para adaptarse a ellos.

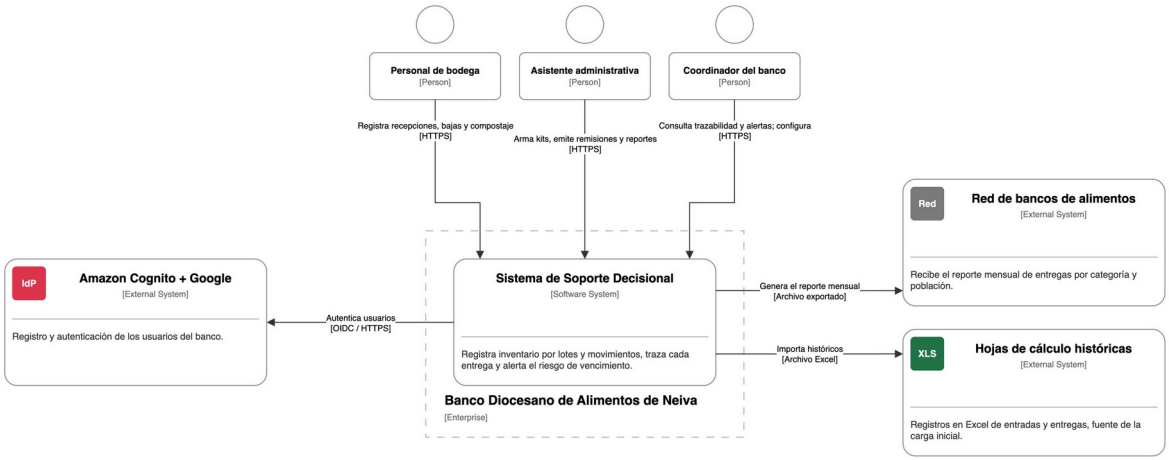
1.2 Objetivo

Diseñar un sistema de soporte a decisiones que dé trazabilidad a las donaciones y anticipe el riesgo de vencimiento, con tres objetivos de negocio:

- Reducir el desperdicio: medir las pérdidas en recepción y en bodega y priorizar la salida de lo que vence primero. Medida: kg desechados y dados de baja por mes, por donante y por categoría, disponibles en el sistema.
- Trazar cada producto del donante a la organización. Medida: 100 % de los kg entregados vinculados a una entrada registrada.
- Dejar de redigitar registros y de armar a mano el reporte mensual. Medida: reporte mensual generado sin volver a digitar datos.

1.3 Vista de Contexto C4

El personal de bodega, la asistente administrativa y el coordinador del banco usan el sistema. La autenticación se delega en Amazon Cognito con Google como proveedor de identidad. El intercambio con la red de bancos y con las hojas de cálculo históricas se hace por archivos. EatCloud, las organizaciones beneficiarias y los donantes no interactúan con el sistema.



2. Drivers Arquitectónicos

2.1 Funcionalidades Principales

- UC-01 Registrar recepción: el personal de bodega registra la donación con el soporte del donante y la selección en aprovechado y desechado.
- UC-02 Consultar existencias y trazabilidad: el coordinador consulta las existencias por producto, presentación y vencimiento, y el origen de cada producto entregado.
- UC-03 Armar kit y emitir remisión: la asistente administrativa arma un kit sacando primero lo que vence antes (FEFO) y emite la remisión para la organización.
- UC-04 Alertar riesgo de vencimiento: el sistema calcula el riesgo de vencimiento de los lotes y el coordinador recibe las alertas.
- UC-05 Registrar bajas y compostaje: el personal de bodega registra las bajas en bodega y las salidas a compostaje.
- UC-06 Generar reporte mensual: la asistente administrativa genera el reporte mensual por categoría y población para la red de bancos.
- UC-07 Importar históricos: el coordinador carga los históricos en Excel.
- UC-08 Gestionar catálogo: el coordinador registra productos, categorías, presentaciones y la vida útil por categoría.
- UC-09 Vincular organización: la asistente administrativa registra la vinculación de una organización beneficiaria y sus etapas.
- UC-10 Configurar kits por programa: el coordinador define la composición de los kits de cada programa.
- UC-11 Gestionar usuarios y roles: el coordinador da de alta a los usuarios del banco y les asigna sus roles.

La funcionalidad primaria son UC-01 a UC-04, UC-06 a UC-08 y UC-10. Quedan fuera de alcance la cartera (aporte solidario y cuentas por cobrar), el apoyo a la distribución equitativa y la operación con otros bancos.

2.2 Restricciones

- CON-1: es un prototipo de maestría limitado a los flujos críticos (trazabilidad y riesgo de vencimiento), que se valida con escenarios de calidad y no entra a producción.
- CON-2: el equipo son tres personas y el plazo es el cronograma del trabajo de grado.
- CON-3: como entidad sin ánimo de lucro, la operación debe tener costo bajo.
- CON-4: se tratan datos personales de representantes y de la población atendida, por lo que aplica la Ley 1581 de 2012.
- CON-5: el sistema debe generar documentos equivalentes a los formatos institucionales FT-ADM, como la remisión firmada.
- CON-6: el riesgo de vencimiento se calcula inicialmente con reglas por categoría y vida útil; el modelo predictivo se incorpora después.
- CON-7: no hay integración con servicios externos (por ejemplo EatCloud); el intercambio se hace por registro manual o por archivos.

2.3 Supuestos

- AS-1: la carga es baja: del orden de 30 a 90 toneladas al mes, unas 20 remisiones por día y menos de 10 usuarios a la vez.
- AS-2: la bodega tiene internet estable en horario operativo.
- AS-3: no existe un histórico digital de entradas con fecha de vencimiento.
- AS-4: la panadería y el fruver llegan sin fecha de vencimiento y se estiman por la vida útil de su categoría.

- AS-5: los históricos en Excel tienen nombres de producto no normalizados y estructuras que cambian entre meses.
- AS-6: si el modelo tiene éxito, el sistema podrá operar en otros bancos de alimentos, cada uno con sus datos, usuarios y roles aislados (proyección; fuera del alcance).

2.4 Atributos de Calidad

ID	QA-01 — Origen de un producto entregado
Atributo de Calidad	Trazabilidad › Procedencia de lo entregado (A, A)
Estímulo	Consulta el origen de un producto entregado a una organización
Fuente del Estímulo	Coordinador del banco
Ambiente	Operación normal
Artefacto	Registro de lotes y movimientos de inventario
Response	Muestra donante, fecha de recepción, soporte, selección y remisión
Medida de la Respuesta	Menos de 5 s

ID	QA-02 — Corrección sin borrar historia
Atributo de Calidad	Trazabilidad › Inmutabilidad del historial (A, M)
Estímulo	Corrige la cantidad de un lote
Fuente del Estímulo	Personal de bodega
Ambiente	Operación normal
Artefacto	Registro de movimientos
Response	El ajuste queda como un movimiento nuevo y el original se conserva
Medida de la Respuesta	0 registros borrados o sobrescritos

ID	QA-03 — Recepción sin análisis predictivo
Atributo de Calidad	Disponibilidad › Aislamiento de fallos del análisis (A, M)
Estímulo	El componente de análisis predictivo deja de responder
Fuente del Estímulo	Falla interna

Ambiente	Jornada de recepción, sábados incluidos
Artefacto	Módulos de recepción y distribución
Response	Recepción y remisiones siguen operando; las alertas pendientes se calculan cuando el componente se recupera
Medida de la Respuesta	0 recepciones bloqueadas por una falla del análisis

ID	QA-04 — Cambio de reglas por parámetros
Atributo de Calidad	Mantenibilidad › Configurabilidad de reglas de negocio (A, M)
Estímulo	Cambia la composición de un kit o la vida útil de una categoría
Fuente del Estímulo	Coordinador del banco
Ambiente	Operación normal
Artefacto	Catálogo y definición de kits
Response	Aplica el cambio por parámetros, sin tocar código ni desplegar
Medida de la Respuesta	Menos de 1 hora

ID	QA-05 — Reemplazo del modelo de predicción
Atributo de Calidad	Mantenibilidad › Reemplazabilidad del modelo de predicción (M, M)
Estímulo	Reemplaza el modelo de reglas por uno predictivo
Fuente del Estímulo	Equipo de desarrollo
Ambiente	Mantenimiento
Artefacto	Componente de análisis
Response	El cambio queda contenido en el componente de análisis; inventario y distribución no se modifican
Medida de la Respuesta	0 cambios fuera del componente de análisis

ID	QA-06 — Reporte mensual sin redigitar

Atributo de Calidad	Interoperabilidad › Intercambio con la red de bancos (A, M)
Estímulo	Genera el reporte mensual
Fuente del Estímulo	Asistente administrativa
Ambiente	Cierre de mes
Artefacto	Componente de reportes
Response	Exporta kg por categoría y población en el formato vigente
Medida de la Respuesta	0 datos redigitados

ID	QA-07 — Importación de históricos
Atributo de Calidad	Interoperabilidad › Incorporación de datos históricos (M, A)
Estímulo	Carga un histórico en Excel
Fuente del Estímulo	Coordinador del banco
Ambiente	Puesta en marcha
Artefacto	Importador
Response	Valida, normaliza nombres e informa inconsistencias
Medida de la Respuesta	100 % de las filas importadas o reportadas con su error

ID	QA-08 — Adaptación a un segundo banco
Atributo de Calidad	Flexibilidad › Adaptabilidad (B, M)
Estímulo	Habilita un segundo banco de alimentos
Fuente del Estímulo	Equipo de desarrollo
Ambiente	Mantenimiento
Artefacto	Modelo de datos y control de acceso por banco
Response	El nuevo banco opera con sus datos, usuarios y roles aislados mediante configuración
Medida de la Respuesta	0 cambios de código en inventario, distribución y análisis

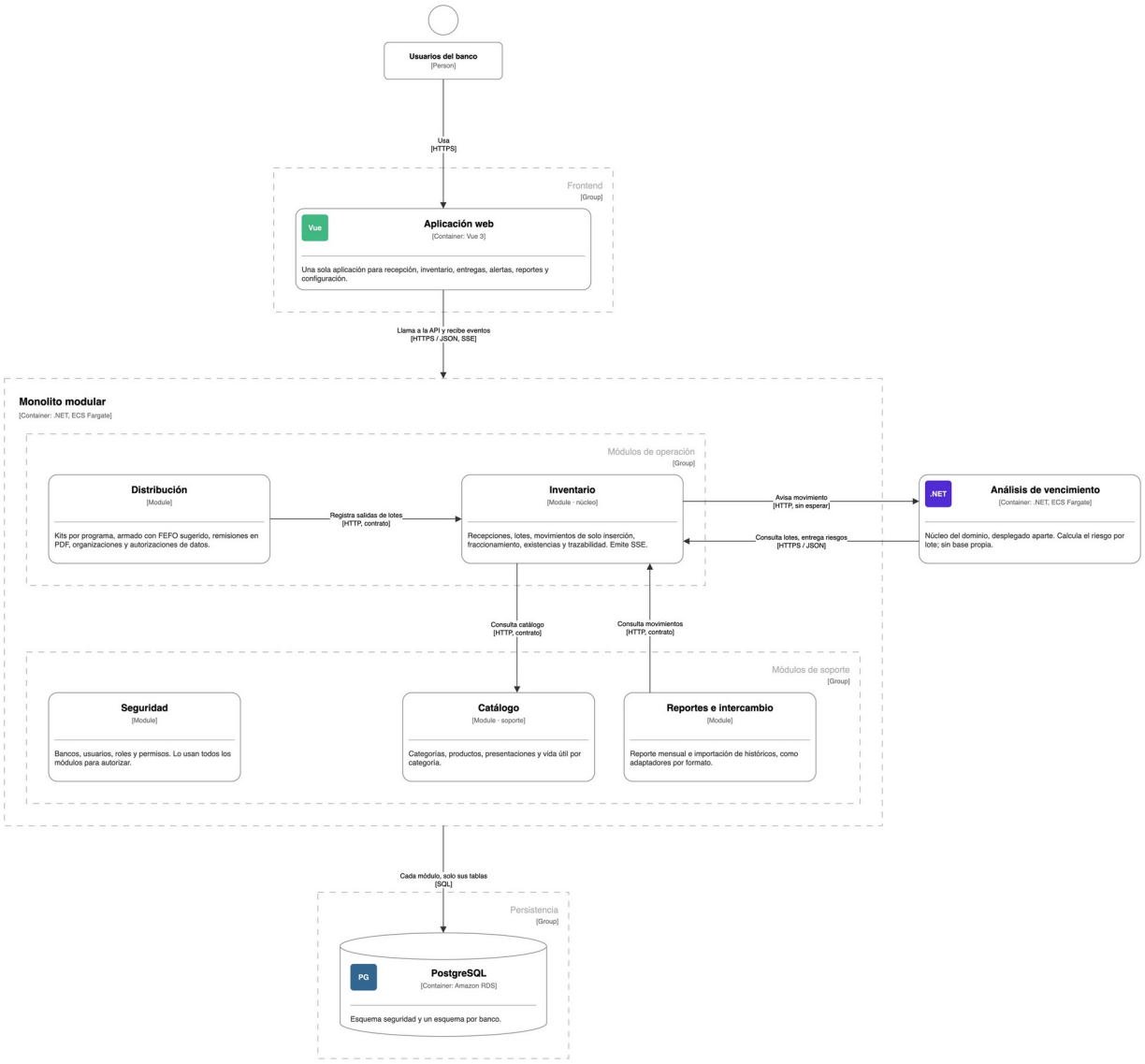
ID	QA-09 — Inventario actualizado en pantalla
Atributo de Calidad	Usabilidad › Actualización en tiempo real (M, B)
Estímulo	Registra una entrada o salida de inventario
Fuente del Estímulo	Personal de bodega o asistente administrativa
Ambiente	Operación normal
Artefacto	Módulo de inventario y pantallas de inventario
Response	Las pantallas de inventario abiertas por los usuarios del mismo banco muestran el cambio sin recargar
Medida de la Respuesta	Menos de 2 s

3. Decisiones de Arquitectura

3.1 Estilo Arquitectónico: Monolito Modular

El sistema se despliega como una sola aplicación .NET dividida en módulos por contexto del dominio. Cada módulo expone su funcionalidad a los demás mediante interfaces y se comunica con ellos por HTTP, sin acceder a los datos de otro módulo. Un solo despliegue mantiene bajo el esfuerzo de un equipo de tres personas (CON-2) y el costo de operación (CON-3), y basta para la carga esperada. Los límites entre módulos dejan contenidos los cambios y permitirían extraer un módulo más adelante. La excepción es el módulo de análisis de vencimiento, que se despliega en un contenedor aparte para que su falla no afecte la operación (QA-03) y para poder reemplazar el modelo de reglas por uno predictivo, incluso en otro lenguaje (QA-05).

Los módulos son Inventario (núcleo del dominio: recepción, lotes, movimientos, bajas, compostaje, existencias y trazabilidad), Análisis de vencimiento (núcleo del dominio: riesgo y alertas), Distribución (kits por programa, armado con FEFO, remisiones, organizaciones y autorizaciones de datos), Catálogo (módulo de soporte del que dependen los demás), Reportes e intercambio (reporte mensual e importación de históricos) y Seguridad (bancos, usuarios, roles y permisos).



3.2 Patrones

3.2.1 Clean Architecture

Cada módulo se organiza en capas de dominio, aplicación, infraestructura y presentación, y las dependencias apuntan hacia el dominio. La persistencia, los formatos de archivo y el proveedor de identidad quedan en la infraestructura, detrás de interfaces. Atiende QA-05, QA-06 y QA-07.

3.2.2 Domain-Driven Design

Los módulos corresponden a contextos delimitados que salen del Event Storming. Las reglas críticas, como el FEFO al armar kits y el lote con sus movimientos, viven en agregados del dominio. Atiende QA-01 y QA-02.

3.2.3 CQRS

Se separan los comandos, que registran movimientos, de las consultas de existencias y trazabilidad. Las consultas se pueden optimizar con vistas de lectura sin tocar el modelo de escritura. Atiende QA-01 y QA-02.

3.2.4 Unit of Work

Cada comando confirma en una sola transacción todos sus cambios, por ejemplo una remisión con los movimientos de salida de sus lotes, de modo que no queden registros a medias. Atiende QA-01 y QA-02.

3.2.5 Comunicación entre Módulos mediante Contrato
Cada módulo publica un contrato, es decir una interfaz y sus DTO, y los demás módulos lo consumen por HTTP. Ningún módulo consulta las tablas de otro. Atiende QA-05.

3.2.6 Análisis Disparado por Movimiento con Recálculo Periódico
Cada vez que se confirma un movimiento de inventario, el monolito avisa al módulo de análisis por HTTP sin esperar su respuesta, y el análisis recalcula el riesgo de ese lote. El aviso se envía después de confirmar la transacción, desde un proceso en segundo plano y con timeout corto, de modo que la recepción nunca espera al análisis. El análisis también recalcula todos los lotes vigentes una vez al día, porque el riesgo cambia con el paso del tiempo, y cada vez que arranca, con lo que recupera lo que no calculó mientras estuvo caído. Un cambio de vida útil de una categoría también dispara el recálculo de sus lotes. Atiende QA-03, QA-04 y QA-05.

3.2.7 Notificaciones en Tiempo Real con Server-Sent Events
Al confirmarse un movimiento de inventario (recepción, baja, compostaje o salida por remisión), el monolito emite un evento SSE a los navegadores conectados del mismo banco, y las pantallas de inventario se refrescan sin recargar. Cuando el análisis entrega resultados nuevos, también se emite un evento y se refresca el panel de alertas. Atiende QA-09.

3.2.8 Control de Acceso Basado en Roles
Cada usuario pertenece a un banco y puede tener varios roles; cada rol agrupa permisos definidos por el sistema, como registrar recepción o emitir remisión. Los roles son propios de cada banco. Los datos personales de representantes y de la población atendida solo son visibles con el permiso correspondiente. Atiende CON-4 y QA-08.

3.2.9 Registro de Autorizaciones de Tratamiento de Datos
Cada titular de datos personales tiene registrada su autorización: fecha, medio, finalidades autorizadas, versión de la política de tratamiento y soporte escaneado. Como regla del dominio de Distribución, no se guardan los datos personales de un titular sin una autorización registrada. Atiende CON-4.

3.2.10 Minimización de Datos Personales
La población atendida se registra como cifras por organización y programa, no como personas identificadas, lo que basta para el reporte mensual. Queda sujeta a confirmación del banco. Atiende CON-4 y QA-06.

3.3 Base de Datos
PostgreSQL en una sola base de datos, con un esquema seguridad compartido (bancos, usuarios, roles y permisos) y un esquema por banco con sus datos operativos (ADR-002). El esquema del banco se fija en cada transacción a partir del banco del usuario autenticado, para que una conexión reutilizada no exponga datos de otro banco. Los movimientos de inventario son de solo inserción: los ajustes se registran como movimientos nuevos y la base bloquea UPDATE y DELETE sobre ellos (ADR-004). La base se cifra en reposo.

3.4 Servicios Cloud Seleccionados
El proveedor es AWS, en la región us-east-1.

- Amazon ECS con AWS Fargate: ejecuta el monolito .NET y el módulo de análisis como dos tareas separadas, sin administrar servidores.
- Application Load Balancer: entrada HTTPS al monolito, con el tiempo de espera ampliado para mantener abiertas las conexiones SSE.
- NAT Gateway: salida a internet de las tareas en la subred privada (llaves JWKS de Cognito e imágenes de contenedores), sin exponerlas.
- Amazon RDS for PostgreSQL: base administrada, en una instancia pequeña, con respaldos automáticos y cifrado en reposo.
- Amazon S3: almacenamiento cifrado de remisiones en PDF, archivos Excel importados, reportes exportados y soportes de autorización.

- Amazon S3 y Amazon CloudFront: publicación del frontend Vue 3 como sitio estático; CloudFront es el único punto de entrada de los usuarios.
- Amazon Cognito: registro y autenticación de usuarios, con Google como proveedor de identidad.

3.5 Stack Tecnológico

Lista de herramientas y tecnologías requeridas para la correcta implementación del sistema.

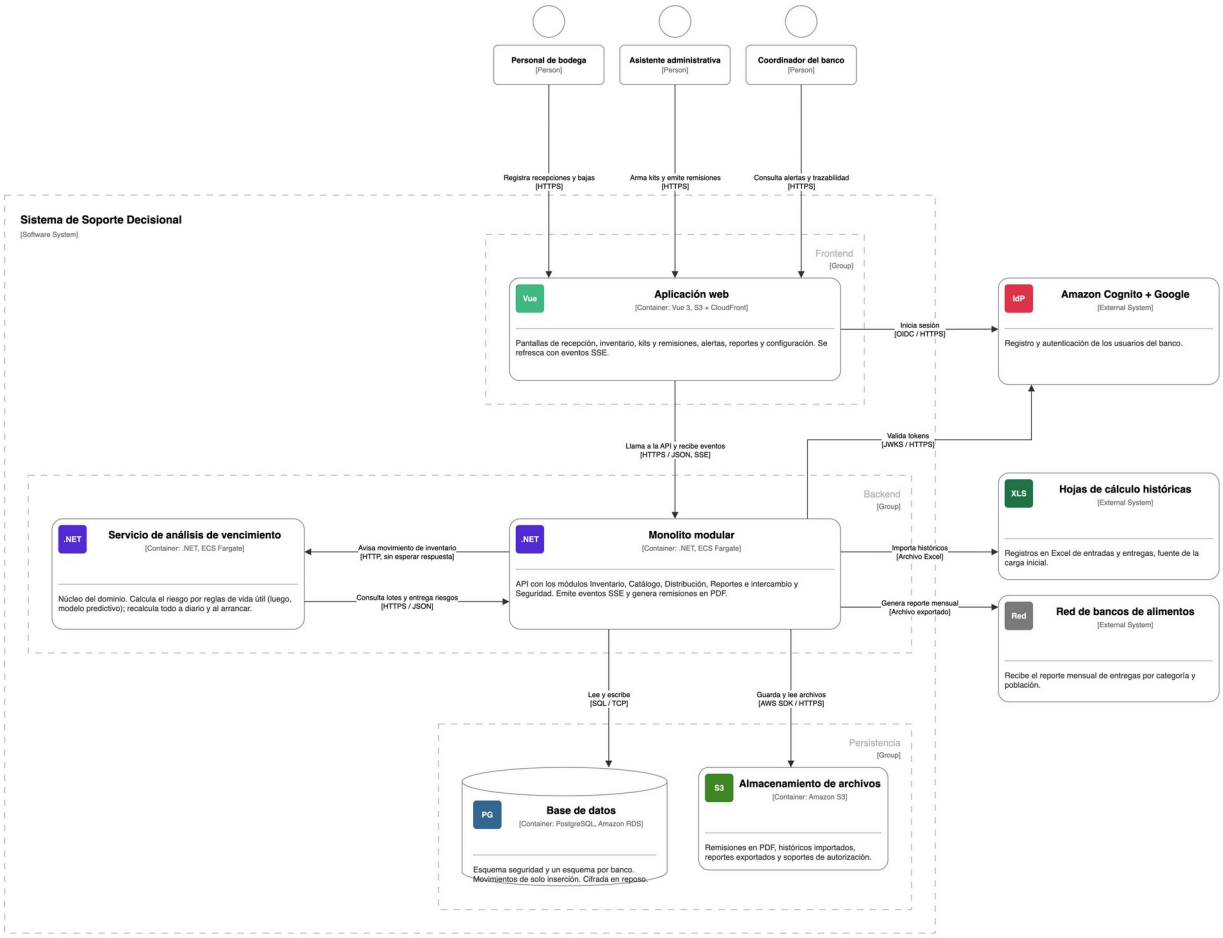
Categoría	Herramientas
Frontend	Vue 3
Backend	C# / .NET
Generación de PDF	QuestPDF
Base de datos	PostgreSQL
Nube	AWS
Pruebas de API	Postman
Desarrollo asistido por IA	OpenCode, Claude Code

4. Definición de Arquitectura

4.1 Vista Funcional

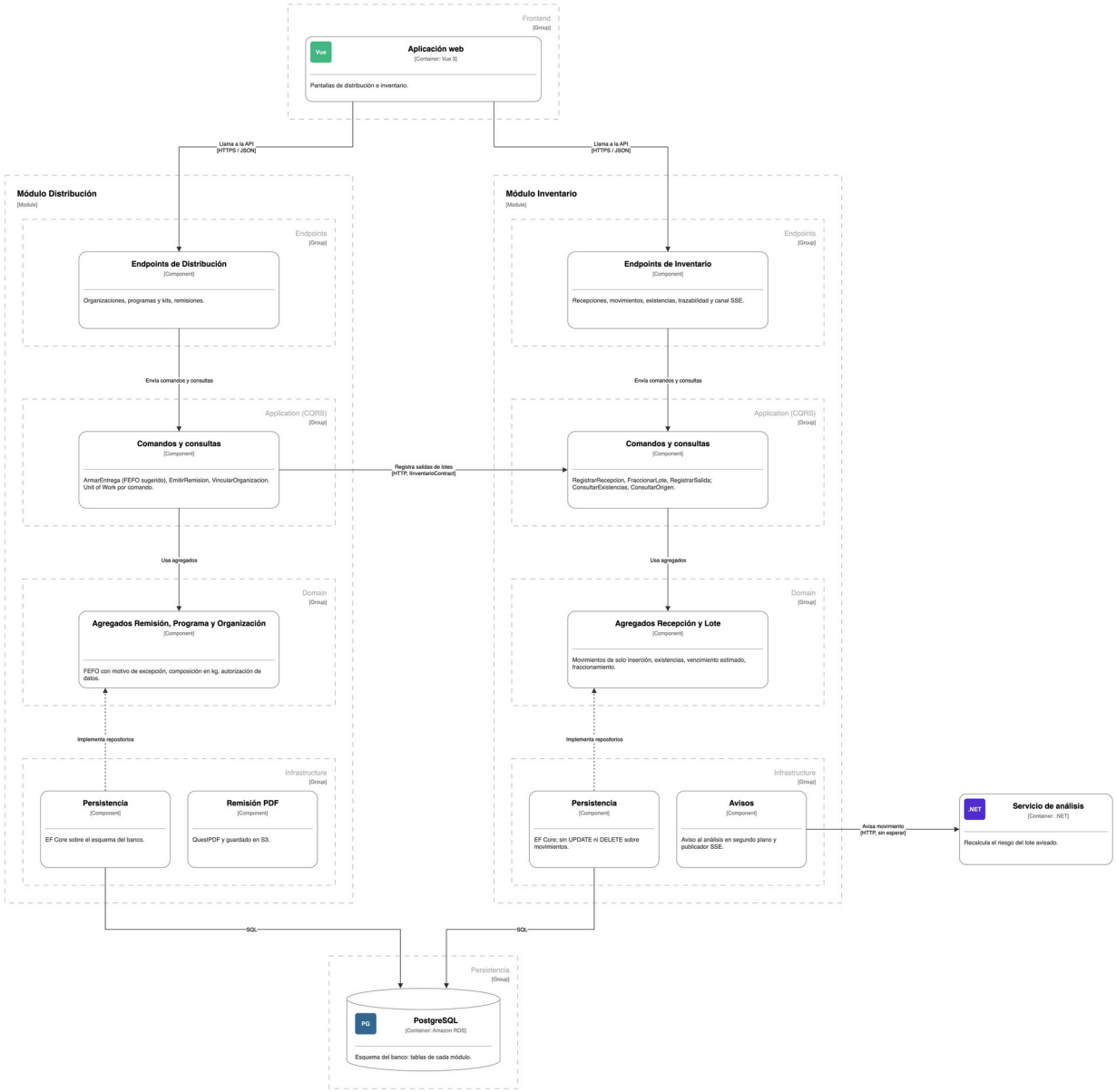
4.1.1 Diagrama de contenedores

La aplicación web en Vue 3 consume la API del monolito modular y recibe sus eventos SSE. El monolito avisa al servicio de análisis cada movimiento sin esperar respuesta, y el análisis consulta los lotes vigentes y entrega los riesgos. El monolito persiste en PostgreSQL y guarda archivos en S3; el análisis no tiene base propia.



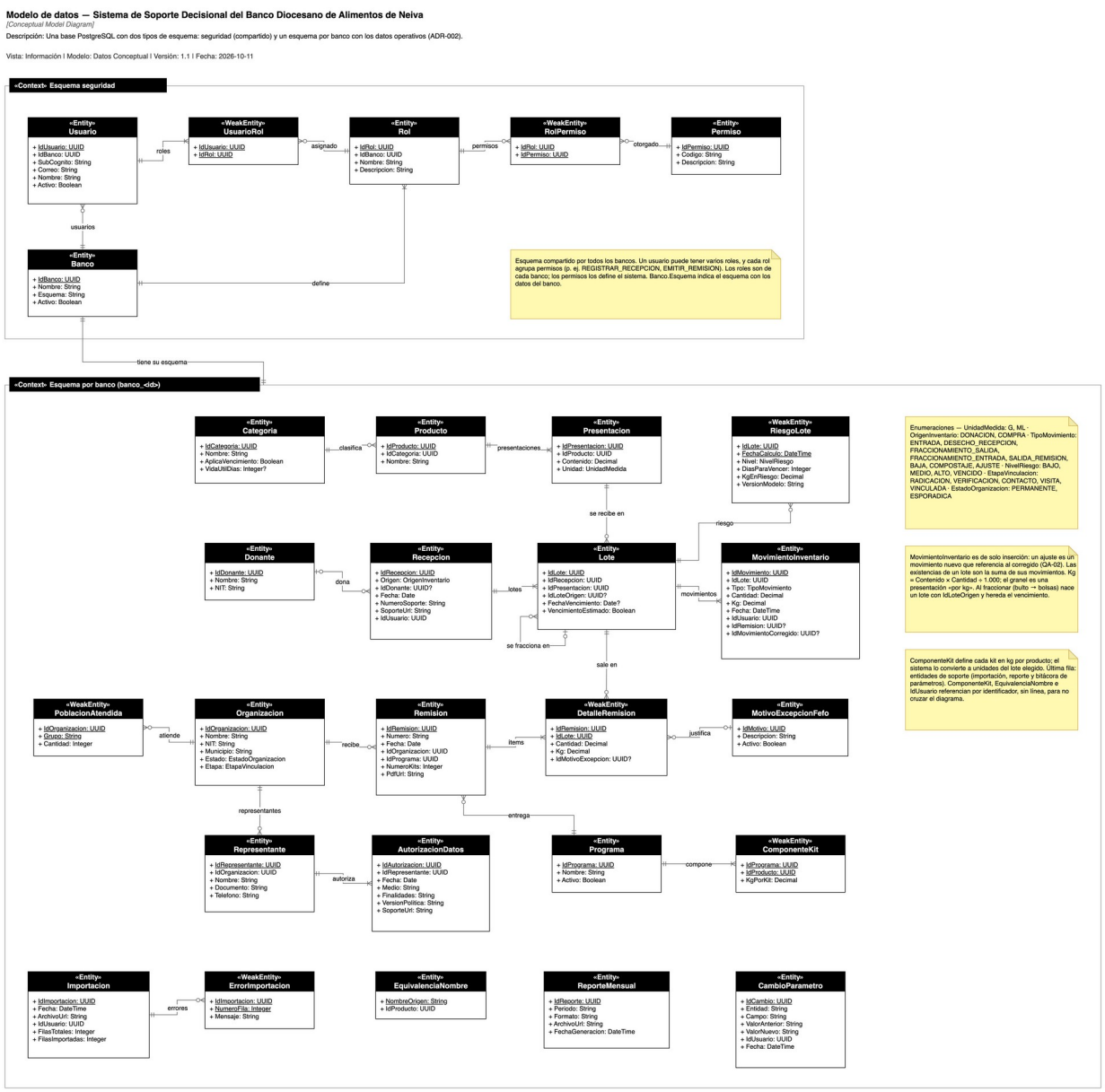
4.1.2 Diagrama de componentes

El diagrama muestra los dos módulos que concentran la trazabilidad, Distribución e Inventario, cada uno con sus capas de Clean Architecture. Distribución registra las salidas de lotes a través del contrato del módulo de Inventario, por HTTP. La infraestructura implementa los repositorios del dominio, y el componente de avisos de Inventario notifica al servicio de análisis y publica los eventos SSE. Los demás módulos siguen la misma estructura.



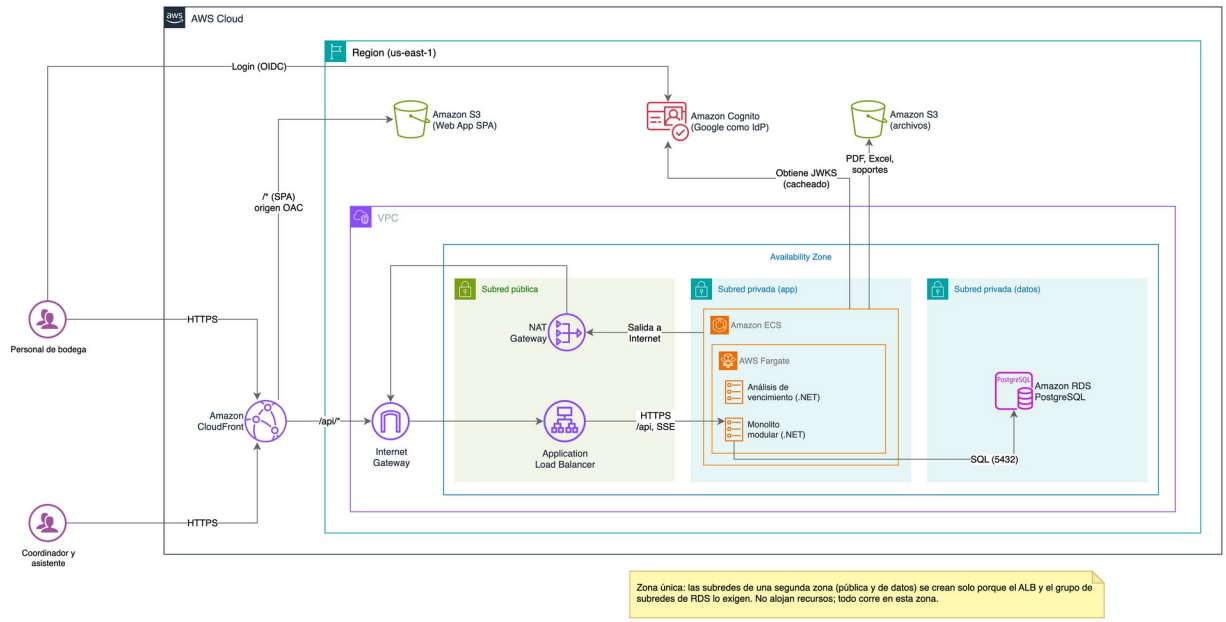
4.2 Vista de Información (Base de Datos)

El modelo conceptual tiene dos tipos de esquema. El esquema seguridad, compartido, guarda bancos, usuarios, roles y permisos; un usuario puede tener varios roles y cada rol agrupa permisos. El esquema de cada banco guarda los datos operativos alrededor del lote: el donante entrega una recepción; cada renglón es un lote de una presentación del catálogo, con su vencimiento real o estimado; cada lote acumula movimientos de solo inserción (entrada, desecho en recepción, fraccionamiento, salida por remisión, baja, compostaje y ajuste) y tiene un riesgo calculado. Los lotes salen hacia una organización en una remisión, cuyos detalles indican de qué lote salió cada cantidad. Al fraccionar un bulto nace un lote nuevo que apunta a su lote de origen y hereda el vencimiento, de modo que la trazabilidad no se corta. Los kilos se calculan como contenido de la presentación por cantidad, y lo que llega a granel se maneja como una presentación por kilo.



4.3 Vista de Despliegue

Los usuarios entran solo por CloudFront, que sirve la aplicación web desde un bucket S3 privado y envía las peticiones /api/* al Application Load Balancer de la subred pública. El monolito y el análisis corren como tareas Fargate en la subred privada de aplicación, y RDS PostgreSQL en la subred privada de datos. El NAT Gateway da salida a internet a las tareas. El prototipo corre en una sola zona de disponibilidad; las subredes de una segunda zona se crean solo porque el ALB y RDS lo exigen.



4.4 Vista de Desarrollo (Paquetes)

La solución se organiza por módulo y, dentro de cada módulo, por capa. Cada módulo publica su contrato en un proyecto Contracts, que es lo único que los demás módulos pueden referenciar.

```
food-bank-dss/
├── src/
│   ├── backend/
│   │   ├── FoodBank.Host/              arranque, autenticación, multitenancy, SSE
│   │   ├── FoodBank.Shared/            kernel común, Unit of Work, resolución de esquema
│   │   └── Modules/
│   │       ├── Inventario/
│   │       │   ├── Inventario.Api/
│   │       │   ├── Inventario.Application/
│   │       │   ├── Inventario.Domain/
│   │       │   ├── Inventario.Infrastructure/
│   │       │   └── Inventario.Contracts/
│   │       ├── Distribucion/           misma estructura por capas
│   │       ├── Catalogo/               misma estructura por capas
│   │       ├── Reportes/               misma estructura; adaptadores por formato
│   │       └── Seguridad/              misma estructura por capas
│   │   ├── analisis/
│   │   │   ├── Analisis.Api/
│   │   │   ├── Analisis.Application/
│   │   │   ├── Analisis.Domain/        reglas por vida útil; luego modelo predictivo
│   │   │   └── Analisis.Infrastructure/
│   │   └── frontend/
│   │       └── web/                    Vue 3
│   │           └── src/
│   │               ├── modules/        inventario, distribucion, catalogo, reportes, seguridad
│   │               └── shared/         cliente API, SSE, autenticación
├── tests/                              pruebas por módulo y de aislamiento entre bancos
└── docs/arquitectura/                  drivers, decisiones, diagramas y validación
```

5. ADRs

Se documentan aquí únicamente las decisiones que implicaron un trade-off real entre alternativas viables. Otras decisiones (impuestas por el contexto o restricciones) están en la sección 3.

ID	**ADR-001:** Estilo arquitectónico
Contexto	Un equipo de tres personas con el plazo del trabajo de grado (CON-2), costo bajo (CON-3) y carga baja (AS-1); al mismo tiempo, una falla del análisis no debe bloquear la operación (QA-03) y el modelo de predicción debe poder reemplazarse (QA-05).
Alternativas	**Alternativa 1: Microservicios** Un servicio desplegable por cada contexto del dominio, cada uno con su base de datos. Ventajas • Aislamiento de fallos y despliegue independiente por servicio. Desventajas • Mucha infraestructura y coordinación para tres personas. • Más costo de operación. • Consultas de trazabilidad (QA-01) repartidas entre servicios. **Alternativa 2: Monolito en capas** Una aplicación con capas técnicas (presentación, negocio, datos), sin límites por dominio. Ventajas • Lo más simple de construir y desplegar. Desventajas • El análisis queda acoplado al inventario (incumple QA-05) y una falla suya afecta la recepción (QA-03). **Alternativa 3: Monolito modular con el análisis en un contenedor aparte** *(elegida)* Una aplicación .NET con módulos por contexto del dominio, comunicados por HTTP con interfaces; el análisis se despliega en un contenedor separado. Ventajas • Un solo despliegue principal. • Límites claros entre módulos. • La falla del análisis queda aislada y su modelo se reemplaza sin tocar el resto. Desventajas • Dos despliegues en vez de uno. • La disciplina de no cruzar módulos depende del equipo.
Decisión	Monolito modular, con el análisis de vencimiento en un contenedor aparte.
Consecuencias	Los contratos entre módulos deben definirse y respetarse desde el inicio. Si un módulo se extrae después, sus contratos HTTP ya existen.

ID	**ADR-002:** Aislamiento de datos por banco (multitenant)

Contexto	El sistema debe quedar preparado para habilitar otro banco con sus datos, usuarios y roles aislados (QA-08, AS-6), sin costo de operación adicional (CON-3) y protegiendo datos personales (CON-4).
Alternativas	**Alternativa 1: Base compartida con identificador de banco en cada tabla** Todas las tablas llevan una columna de banco y cada consulta filtra por ella. Ventajas • Un solo esquema que migrar. • Lo más barato. Desventajas • Un filtro olvidado expone datos de otro banco. • El aislamiento depende del código. **Alternativa 2: Una base de datos por banco** Cada banco tiene su propia base de datos. Ventajas • El mayor aislamiento. Desventajas • Más instancias o bases que operar, respaldar y migrar. • Más costo. **Alternativa 3: Una base con un esquema seguridad compartido y un esquema por banco** *(elegida)* El esquema seguridad guarda bancos, usuarios y roles; cada banco tiene su propio esquema con sus datos operativos, seleccionado según el banco del usuario. Ventajas • Aislamiento a nivel de esquema sin multiplicar bases. • Habilitar un banco es crear su esquema por configuración. Desventajas • Las migraciones deben aplicarse a cada esquema de banco. • Los reportes entre bancos requieren consultas a varios esquemas.
Decisión	PostgreSQL con un esquema seguridad y un esquema por banco.
Consecuencias	Se necesita un mecanismo que resuelva el esquema a partir del banco del usuario autenticado y un proceso de migración que recorra todos los esquemas. El esquema se fija en cada transacción a partir del banco del usuario autenticado, nunca se deja en la conexión, para que una conexión reutilizada del pool no exponga datos de otro banco; pruebas automatizadas verifican el aislamiento. En el prototipo opera un solo banco.

ID	**ADR-003:** Comunicación con el módulo de análisis
Contexto	El riesgo de vencimiento debe actualizarse con los movimientos de inventario y con el paso del tiempo (UC-04), pero una falla del análisis no puede bloquear la recepción y lo pendiente debe calcularse al recuperarse (QA-03); los módulos se comunican por

	HTTP.
Alternativas	**Alternativa 1: Llamada HTTP síncrona en cada movimiento** El inventario llama al análisis y espera su respuesta antes de confirmar. Ventajas • El riesgo queda calculado al instante. Desventajas • Si el análisis falla, la recepción se bloquea. • Incumple QA-03. **Alternativa 2: Outbox con entrega HTTP y reintentos** El inventario guarda el aviso en una tabla dentro de su transacción y un proceso en segundo plano lo entrega con reintentos. Ventajas • Ningún aviso se pierde. Desventajas • Una tabla, un proceso y una lógica de reintentos más que construir. • No cubre el paso del tiempo por sí sola. **Alternativa 3: Mensajería con Amazon SQS** El inventario publica los cambios en una cola que consume el análisis. Ventajas • Entrega garantizada y desacople total. Desventajas • Otro servicio que operar y pagar. • Rompe la convención de comunicación por HTTP. **Alternativa 4: Aviso HTTP sin espera más recálculo diario y al arrancar** *(elegida)* El monolito avisa al análisis sin esperar respuesta en cada movimiento; el análisis recalcula todos los lotes vigentes una vez al día y cada vez que arranca. Ventajas • La recepción nunca espera al análisis. • Lo perdido se recupera solo. • Cubre el paso del tiempo. • Sin infraestructura adicional. Desventajas • Si el análisis está caído, sus alertas se atrasan hasta que se recupere.
Decisión	Aviso HTTP sin espera por movimiento, con recálculo completo diario y al arrancar.
Consecuencias	El recálculo completo debe tolerar el volumen de lotes vigentes (AS-1). Las alertas se publican en pantalla por SSE (ADR-008).

ID	**ADR-004:** Registro del inventario
Contexto	Cada kg entregado debe vincularse a su entrada (QA-01) y las correcciones no pueden borrar historia (QA-02); se usa CQRS para separar escrituras y consultas.
Alternativas	**Alternativa 1: Saldos actualizables con tabla de auditoría** Cada lote guarda su cantidad actual y los cambios se copian a una bitácora. Ventajas • Consultas de existencias directas. Desventajas • La historia es secundaria y puede divergir del saldo. • La trazabilidad depende de la bitácora. **Alternativa 2: Event Sourcing** El estado de cada lote se reconstruye a partir de su secuencia de eventos. Ventajas • Historia completa por diseño. Desventajas • Curva de aprendizaje y complejidad altas para el plazo (CON-2). • Requiere proyecciones para todas las consultas. **Alternativa 3: Movimientos inmutables de solo inserción** *(elegida)* Cada entrada, salida, baja o ajuste es un movimiento nuevo sobre un lote; nunca se actualiza ni se borra. Las existencias se obtienen de los movimientos, con vistas de lectura (CQRS) si hace falta rendimiento. Ventajas • Trazabilidad e historia sin piezas adicionales. • Modelo relacional conocido por el equipo. Desventajas • Las existencias requieren sumar movimientos o mantener vistas de lectura.
Decisión	Movimientos inmutables de solo inserción.
Consecuencias	La base no debe permitir UPDATE ni DELETE sobre la tabla de movimientos (por permisos o por triggers). QA-01 (menos de 5 s) se verifica con vistas de lectura si la suma directa no alcanza.

ID	**ADR-005:** Reglas de negocio configurables
Contexto	La composición de los kits y la vida útil por categoría cambian con frecuencia y deben aplicarse sin tocar código ni desplegar (QA-04); el riesgo se calcula inicialmente con reglas por categoría (CON-6).
Alternativas	**Alternativa 1: Reglas en el código**

	Kits y vida útil definidos como constantes o configuración desplegada. Ventajas • Lo más simple. Desventajas • Cada cambio exige desarrollar y desplegar. • Incumple QA-04. **Alternativa 2: Motor de reglas externo** Un motor de reglas genérico evalúa reglas escritas por el usuario. Ventajas • Máxima flexibilidad. Desventajas • Sobredimensionado para dos tipos de regla. • Más complejidad y aprendizaje. **Alternativa 3: Parámetros editables desde la aplicación** *(elegida)* La vida útil es un atributo de la categoría y la composición de kits es un dato por programa; el coordinador edita la vida útil desde el módulo de catálogo y la composición de kits desde el módulo de distribución. Ventajas • Cambio inmediato sin despliegue. • Simple de construir. Desventajas • Solo cubre las reglas previstas. • Una regla nueva requiere desarrollo.
Decisión	Reglas como parámetros: vida útil en el catálogo y composición de kits en distribución.
Consecuencias	Los cambios de parámetros deben quedar registrados (quién y cuándo) para no perder trazabilidad de por qué un kit tuvo cierta composición.

ID	**ADR-006:** Importación y exportación de archivos
Contexto	El reporte mensual debe salir en el formato vigente, que cambia (QA-06), y los históricos en Excel tienen estructuras variables y nombres no normalizados (QA-07, AS-5); no hay integración con servicios externos (CON-7).
Alternativas	**Alternativa 1: Formatos fijos dentro de los módulos de dominio** El código del reporte y del importador vive junto a la lógica de inventario. Ventajas • Menos piezas. Desventajas • Cada cambio de formato toca el dominio.

	Alternativa 2: Adaptadores por formato en la infraestructura del módulo de reportes e intercambio *(elegida)* Cada formato de entrada o salida es un adaptador detrás de una interfaz (Clean Architecture); el dominio no conoce los formatos. El importador valida, normaliza nombres contra el catálogo y reporta cada fila con su error. Ventajas • Un formato nuevo es un adaptador nuevo, sin tocar el dominio. Desventajas • Cada formato nuevo requiere desarrollo.
Decisión	Adaptadores por formato en la infraestructura del módulo de reportes e intercambio.
Consecuencias	La normalización de nombres requiere una tabla de equivalencias editable por el coordinador.

ID	**ADR-007:** Plataforma de despliegue en AWS
Contexto	Dos contenedores (monolito y análisis) con fallos aislados (QA-03), conexiones SSE largas (QA-09), costo bajo (CON-3) y un equipo pequeño (CON-2).
Alternativas	**Alternativa 1: Una instancia Lightsail o EC2 con Docker Compose** Los contenedores, y opcionalmente PostgreSQL, en un solo servidor. Ventajas • El menor costo. Desventajas • El servidor se administra a mano. • Si cae, cae todo. • Respaldos manuales. **Alternativa 2: AWS App Runner** Contenedores administrados con escalado automático. Ventajas • Muy poca configuración. Desventajas • Corta las peticiones a los 120 s, lo que rompe el SSE (QA-09). **Alternativa 3: Amazon ECS con Fargate, Application Load Balancer y RDS for PostgreSQL** *(elegida)* Una tarea Fargate por contenedor, entrada por ALB y base administrada. Ventajas • Sin servidores que administrar. • Tareas aisladas.

	• Respaldos automáticos. • Soporta SSE. Desventajas • Costo fijo mayor por el ALB y el RDS.
Decisión	ECS Fargate con ALB y RDS for PostgreSQL.
Consecuencias	Se usa una sola tarea del monolito; si se escala a varias, el SSE necesita un canal compartido entre instancias. El prototipo se despliega en una sola zona de disponibilidad; el ALB y el grupo de subredes de RDS exigen subredes en dos zonas, que se crean sin duplicar las tareas. El NAT Gateway agrega un costo fijo mensual (CON-3).

ID	**ADR-008:** Actualización de pantallas en tiempo real
Contexto	Las pantallas de inventario deben mostrar entradas y salidas sin recargar en menos de 2 s (QA-09), y el panel de alertas debe reflejar los resultados del análisis (UC-04).
Alternativas	**Alternativa 1: Consulta periódica desde el navegador (polling)** El frontend pide los datos cada pocos segundos. Ventajas • Lo más simple. • Sin conexiones abiertas. Desventajas • Carga inútil y retraso según el intervalo. **Alternativa 2: WebSockets con SignalR** Canal bidireccional, nativo en .NET. Ventajas • Bidireccional. • Respaldo automático de transporte. Desventajas • La bidireccionalidad no se necesita. • Más piezas que SSE. **Alternativa 3: Server-Sent Events** *(elegida)* Canal unidireccional del servidor al navegador sobre HTTP. Ventajas • Estándar del navegador. • Simple. • Suficiente para avisos. Desventajas • Unidireccional. • Requiere ajustar el tiempo de espera del ALB.

Decisión	SSE, filtrado por banco.
Consecuencias	Al reconectarse, el cliente recarga los datos de la pantalla para no depender de eventos perdidos.

ID	**ADR-009:** Aplicación de FEFO al armar kits
Contexto	El armado de kits debe sacar primero lo que vence antes para reducir el desperdicio (UC-03), y cada kg entregado debe quedar vinculado al lote del que realmente salió (QA-01).
Alternativas	**Alternativa 1: FEFO obligatorio** El sistema asigna automáticamente el lote que vence primero y no permite cambiarlo. Ventajas • Nunca sale un lote nuevo antes que uno viejo. Desventajas • No permite registrar excepciones reales de bodega (lote dañado, lote no encontrado). • El registro puede apartarse de lo ocurrido físicamente y romper la trazabilidad. **Alternativa 2: FEFO sugerido con motivo obligatorio** *(elegida)* El sistema preselecciona el lote que vence primero; si la asistente elige otro, debe indicar un motivo de una lista. Ventajas • El registro coincide con lo ocurrido. • Las excepciones y sus motivos quedan disponibles para el coordinador como información de decisión. Desventajas • Depende de que las excepciones se usen solo cuando corresponde.
Decisión	FEFO como sugerencia del agregado de distribución, con motivo obligatorio cuando se elige otro lote.
Consecuencias	La lista de motivos es configurable en el módulo de distribución; se necesita una consulta de excepciones a FEFO por periodo y motivo.